Consumer Dispute Intake Assistant

Problem 2: AI lawyer | One practical step: turn a person's consumer dispute story into a clear, evidence-ready intake note.

> **The situation.** A person may know what happened but not know which details matter: purchase and delivery dates, payment proof, seller responses, refund status, or warranty records. A scattered story slows the next conversation with a support team or lawyer. This prototype helps organize that story before anyone assesses the legal merits.

The problem I chose, and why

I chose the consumer-dispute part of Problem 2 and narrowed it to **intake preparation** for an Indian purchase or paid service: for example, an order that never arrived, a defective product, or a cancelled event. The project already had a local legal search pipeline, so this is a concrete use of its large legal and case-material collection. A structured note is useful to a person and straightforward for a reviewer to check against the original account.

The output

The user's stated facts, an ordered timeline, important unanswered questions, relevant retrieved legal material when available, and a practical evidence checklist. The assistant does not decide who is legally right or promise a refund.

How the working prototype produces it

1 Prepare sources
Python reads authorized JSON records, splits text into overlapping chunks, and stores them in Chroma.

2 Ask
The person describes the issue in the command-line chat. Conversation context is kept during that run.

3 Find evidence
Chroma retrieves relevant passages. One agent can refine a search; the other searches complementary query formulations.

4 Draft clearly
Groq receives the question and retrieved passages. The prompt asks for plain language, source grounding, and explicit uncertainty.

In the code: `build_knowledge_base.py` chunks at 1,600 characters with 180-character overlap; `agent.py` retrieves three passages per step and allows up to three steps; `agent_multirag.py` uses the original question plus up to three generated searches, retrieving three passages for each; `llm_client.py` calls configured Groq models. The interface is a local CLI, not a deployed web app.

Assumptions and guardrails

What I assume
The person is asking about an Indian consumer purchase or paid service and can share the relevant facts. The local corpus may be incomplete or out of date.

What the system must do
Keep unknowns visible; ask for missing details; use retrieved context; avoid inventing deadlines, terms, evidence, statutes, or outcomes.

This is general legal information, not legal advice. A qualified human should review consequential legal conclusions. Kuku's assignment input pack is confidential and must stay out of public repositories.

Make usefulness measurable

A confident answer is not proof. Check whether the note preserves facts, finds useful material, and helps a real team move faster.

How I would know it works with a real team

Start with a small, consented pilot of **20 anonymized cases** across non-delivery, defective products, and cancelled paid services. For every case, preserve the original user account, retrieved passages, answer, and reviewer score. A lawyer or trained reviewer checks:

Fact fidelity
Were material facts retained? Did the system add any fact the person never gave?

Useful structure
Is the timeline coherent? Are missing questions focused and relevant?

Evidence quality
Do retrieved passages support the legal references, and are sources relevant?

Team value
Would the intake note be usable, and how much preparation time did it save?

Proposed pilot gate: at least 18/20 notes rated usable, no material invented fact in the reviewed sample, and a measured reduction in intake-preparation time. These are targets for a pilot, not results already established. Track unsupported claims and retrieval misses separately; route weak or consequential answers to a human.

How this can scale across Kuku's work

Keep the same intake workflow and add a reviewed source set, issue-specific questions, and an evaluation set for each new legal area. Version and label sources by jurisdiction and date; test updates before re-indexing. At higher volume, add a managed vector store, secure per-case storage, access controls, and a reviewer queue. Monitor retrieval relevance, answer review scores, latency, and model spend. Current model-call volume is bounded: up to three calls for iterative retrieval, or typically a query-generation call plus one answer call in multi-query mode; actual cost depends on tokens and provider pricing.

What works today, and what remains

The prototype runs locally with Groq and produced responses for all three synthetic prompts. One raw answer generalized a six-month limitation concept from unrelated railway material after acknowledging that the corpus did not directly cover event-ticket refunds. This is a grounding failure and shows why source relevance must be checked by a human. The main remaining limitation is deployment: the working CLI has not yet been made available as a hosted app. The proposed pilot above is the next way to measure performance with a real team at broader scale.

- **Working today:** local command-line workflow, Chroma retrieval, Groq responses, and tested consumer-dispute prompts.
- **Deployment:** hosted app, authentication, and persistent case workspace remain future work.
- **Next step:** deploy the workflow and evaluate it with a human-reviewed team pilot.

Prototype

Prototype: github.com/Akash-M-100/AI-LAWYER- (repository README has local setup instructions)